

SignInControlle - Estrutura Arquitetônica

1. Posição do controller

Camada: `interface-adapters/http/controllers`

Dependência permitida do controller:

- contrato genérico `UseCase<SignInInput, SignInOutput>` ou um contrato específico equivalente
- contratos HTTP locais (`HttpRequest` , `HttpResponse` , `Controller`)
- tipos de entrada e saída do caso de uso

Dependências que o controller não deve conhecer:

- `BCryptAdapter`
- `JwtAdapter`
- `PostgresAdapter`
- `UserRepository`
- `BCrypt`
- `JsonWebToken`
- `PostgreSQL`

Essas dependências pertencem ao fluxo interno do caso de uso e da infraestrutura, não ao controller.

2. Fluxo mínimo

`Framework/Route Adapter -> SignInController -> UseCase<SignInInput, SignInOutput> -> SignInUseCase`

No diagrama anexo, isso respeita o fato de que `SignInUseCase` **orquestra** `PasswordCompare` , `LoadUserByEmail` e `AccessTokenGenerator` , enquanto o controller fica apenas na borda HTTP.

Interfaces propostas

`src/interface-adapters/http/protocols/http.ts`

```
export interface HttpRequest<
  TBody = unknown,
  TParams extends Record<string, string | undefined> = Record<string, string | undefined>,
  TQuery extends Record<string, string | undefined> = Record<string, string | undefined>,
  THeaders extends Record<string, string | undefined> = Record<string, string | undefined>,
> {
  readonly body?: TBody
  readonly params: TParams
  readonly query: TQuery
  readonly headers: THeaders
}

export interface HttpResponse<TBody = unknown> {
  readonly statusCode: number
  readonly body: TBody
}
```

`src/interface-adapters/http/protocols/controller.ts`

```
export interface Controller<TRequest, TResponse> {
  handle(request: TRequest): Promise<TResponse>
}
```

src/interface-adapters/http/controllers/sign-in/sign-in-http-dtos.ts

```
export interface SignInHttpRequestBody {
  readonly email?: unknown
  readonly password?: unknown
}

export interface SignInSuccessHttpResponseBody {
  readonly accessToken: string
}

export interface HttpErrorResponseBody {
  readonly message: string
  readonly code?: string
}

export type SignInHttpResponseBody = SignInSuccessHttpResponseBody | HttpErrorResponseBody
```

src/interface-adapters/http/controllers/sign-in/sign-in-controller.contract.ts

```
import type { UseCase } from '#src/application/use-cases/common/use-case'
import type { SignInInput } from '#src/application/use-cases/sign-in/sign-in-input'
import type { SignInOutput } from '#src/application/use-cases/sign-in/sign-in-output'
import type { Controller } from '#src/interface-adapters/http/protocols/controller'
import type { HttpRequest, HttpResponse } from '#src/interface-adapters/http/protocols/http'

import type {
  SignInHttpRequestBody,
  SignInHttpResponseBody,
} from '#src/interface-adapters/http/controllers/sign-in/sign-in-http-dtos'

export type SignInHttpRequest = HttpRequest<SignInHttpRequestBody>

export type SignInHttpResponse = HttpResponse<SignInHttpResponseBody>

export interface SignInControllerDependencies {
  readonly signIn: UseCase<SignInInput, SignInOutput>
}

export interface SignInControllerContract extends Controller<SignInHttpRequest, SignInHttpResponse> {}
```

Responsabilidades do controller

Deve fazer

- validar fronteira HTTP
- garantir presença de `email` e `password`
- garantir que ambos chegam como `string`
- mapear a entrada HTTP para `SignInInput`
- chamar o caso de uso
- devolver `200` com `accessToken` no sucesso
- traduzir erros esperados para `400` ou `401`
- traduzir falhas inesperadas para `500`

Não deve fazer

- consultar banco
- comparar senha
- gerar token
- conhecer adapters concretos
- conter regra de negócio de autenticação
- duplicar validações semânticas que pertencem a `Email`, `Password` ou ao próprio caso de uso

Isso preserva baixo acoplamento e mantém o controller com responsabilidade única, o que também é compatível com a abordagem de testes unitários sem recursos externos.

Observação importante sobre `SignInInput`

Pelo diagrama, `SignInInput` está ligado aos objetos de domínio `Email` e `Password`. Portanto, a implementação concreta do controller pode seguir um destes caminhos:

Opção mais fiel ao diagrama

O controller transforma `body.email` e `body.password` em `Email` e `Password` antes de montar `SignInInput`.

Opção mais pragmática

O controller repassa strings para o caso de uso, e o próprio caso de uso instancia `Email` e `Password`.

Entre as duas, eu prefiro a segunda para este exercício, porque reduz acoplamento do controller com detalhes do domínio e mantém a borda HTTP mais simples. Mas, se o projeto já definiu `SignInInput` com VOs explícitos, então a primeira opção fica mais aderente ao desenho existente.

Roteiro de testes de unidade

Abaixo, adaptei o modelo do arquivo anexo para `SignInController`.

Roteiro de Testes para `SignInController`

1. `SignInController`

Esta é a classe da camada de interface responsável por receber a requisição HTTP de autenticação, validar a fronteira de entrada, mapear os dados para `SignInInput`, invocar o caso de uso de sign in e traduzir o resultado para uma resposta HTTP apropriada. Seu foco não é autenticar diretamente, mas adaptar o transporte HTTP ao contrato da aplicação. A ideia de manter adapters como ponte entre caso de uso e recursos externos é consistente com o material de Clean Architecture.

1.1. `should call sign in use case with email and password from request body`

Deve verificar se o controller extrai `email` e `password` do corpo da requisição e chama o caso de uso com a entrada correspondente.

1.2. `should return 200 and access token when credentials are valid`

Deve garantir que, quando o caso de uso retorna sucesso, o controller responda com status `200` e corpo contendo o token de acesso.

1.3. `should return 400 when request body is missing`

Deve garantir que o controller rejeita a requisição quando o corpo HTTP não é fornecido.

1.4. `should return 400 when email is missing`

Deve verificar que o controller responde com erro de requisição inválida quando `email` não está presente no corpo.

1.5. `should return 400 when password is missing`

Deve verificar que o controller responde com erro de requisição inválida quando `password` não está presente no corpo.

1.6. `should return 400 when email or password are not strings`

Deve garantir que a validação de fronteira rejeita entradas em que `email` ou `password` chegam com tipo incompatível com o contrato HTTP esperado.

1.7. `should not call sign in use case when request is invalid`

Deve verificar que, diante de falha de validação de entrada na borda HTTP, o controller não invoca o caso de uso.

1.8. `should return 401 when credentials are invalid`

Deve garantir que, quando o caso de uso sinaliza credenciais inválidas, o controller traduz adequadamente esse cenário para resposta HTTP `401`.

1.9. `should return 400 when use case rejects malformed email or password`

Deve verificar que, quando o caso de uso ou os value objects de domínio rejeitam `email` ou `password` malformados, o controller traduz esse erro esperado para `400`, sem mascará-lo como erro interno.

1.10. `should return 500 when sign in use case throws unexpected error`

Deve garantir que falhas inesperadas lançadas pelo caso de uso sejam traduzidas para `500`, preservando o isolamento entre a interface HTTP e detalhes internos do erro.

1.11. `should not depend on SignInUseCase concrete class directly`

Deve verificar, em termos arquiteturais, que o controller depende apenas do contrato do caso de uso, como `UseCase<SignInInput, SignInOutput>`, e não da classe concreta `SignInUseCase`.

Test doubles recomendados nos testes do controller

Para esse conjunto, eu usaria:

- **stub** para controlar a saída do caso de uso em cenários de sucesso ou falha esperada
- **spy** para verificar se `execute()` foi chamado com a entrada correta
- **mock** apenas quando a interação for o centro do teste
- evitar fake aqui, porque o controller tem apenas uma dependência principal e o teste deve permanecer pequeno

Essa escolha é coerente com a distinção entre stub, spy e mock e com a recomendação de usar mocks com parcimônia.